

## 2.3.2 Number of Conditions

IF پروسیسر سے True/False پوچھتی ہے اس لیے زیادہ شاخیں کارکردگی گھٹاتی ہیں۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 10 of 24

## 2.3.2 Number of Conditions

### LEARN IT

**Conditions** (IF statements) are decision points. Every condition forces the CPU to evaluate True or False, so excessive branching reduces performance.

- **Computational cost.** Each IF is extra work, even if the rest of the algorithm is simple.
- **Nested conditions.** IF inside IF often means a brute-force approach. It hurts both readability and speed.
- **Short-circuiting** (the optimisation goal). Stop evaluating as soon as the result is known. In a logical AND, if the first part is False the second part is skipped. In a logical OR, if the first part is True the second part is skipped.

### WORKED EXAMPLE

**Worked example: cheap test first.** You only divide if the denominator is not zero. Place the cheap, safe test on the left so Python never evaluates the dangerous part.

```
x = 0
y = 10
if x != 0 and (y / x) > 2:
    print("ratio is large")
else:
    print("skipped safely")
```

### OUTPUT

skipped safely

$x \neq 0$  is False, so AND short-circuits and  $y / x$  is never attempted. If you wrote the division first, the program would crash. Same idea in the soldier algorithm: a False height test skips Step 6.

### REMEMBER IT

Put the **cheapest / most likely False** test first in an AND, and the **most likely True** test first in an OR. That is short-circuiting used on purpose.

### COMMON MISTAKE

**Mistake:** writing a chain of nested IFs when one compound condition would do (as in the velocity example in 2.2.2).

**Fix:** flatten with AND/OR, and rely on short-circuiting instead of extra layers.

### اردو میں سمجھیں

ہر IF پروسیسر سے True/False پوچھتی ہے اس لیے زیادہ شاخیں کارکردگی گھٹاتی ہیں۔ اندرونی IF اکثر brute-force ہوتی ہے۔ شارٹ سرکٹ (Short-circuit) کا مطلب ہے جیسے ہی جواب معلوم ہو جانچ روک دی جائے، مثلاً AND میں پہلی شرط False ہو تو دوسری نہ چلے۔

### CHECK YOURSELF

1. Why do extra IF statements reduce performance?
2. What does nested IF often indicate?
3. In  $x \neq 0$  and  $(y / x) > 2$  with  $x = 0$ , why is there no division-by-zero?

Answers: 1. Each condition must be evaluated True/False by the CPU. 2. A brute-force approach (and it reduces readability and speed). 3. AND short-circuits: the first part is False, so the division is never evaluated.

---

Next lecture: 2.3.3 Number of Repetitions